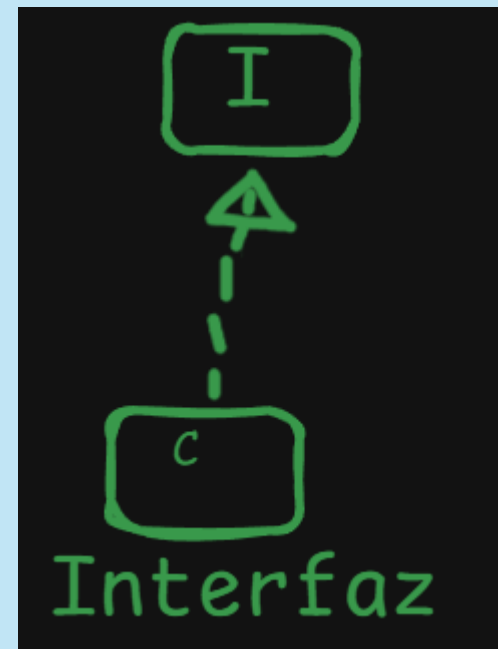
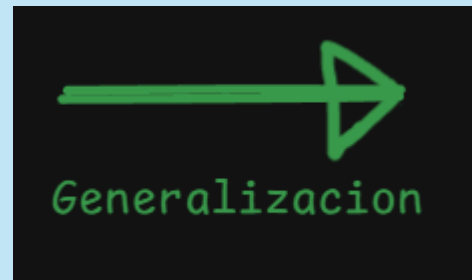
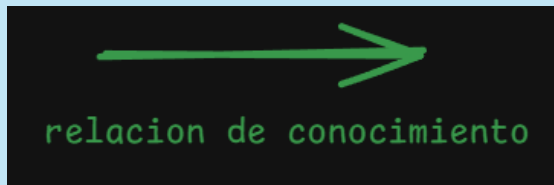


002



2. ¿Qué es un HotSpot?

Es lo que el desarrollador puede personalizar.

Puede ser una clase o un método.

Ejemplos

```
</> Java  
class BluetoothDevice extends Device
```

Clase HotSpot

```
@Override  
void readSignal(){  
}
```

Método HotSpot

Siempre que vos tengas que escribir código para completar el framework, estás trabajando sobre un HotSpot.

En los ejercicios de la facultad **casi siempre** esa regla funciona, porque suelen asociar:

- `extends` → White Box.
- `implements` o composición → Black Box.

Pero conceptualmente, **White Box** y **Black Box** describen el mecanismo de extensión del framework, no el tipo de la clase en sí. Lo importante es **cómo el usuario extiende el framework**. Por eso, en un parcial, mirá primero si la extensión se hace por **herencia** (`extends`) o por **implementación/composición** (`implements`, `register`, `set`, `add`). Ese criterio es el que normalmente esperan que identifiques.

White Box

Palabras clave

- `extends`
- clase abstracta
- herencia
- sobrescribir métodos (`@Override`)

Black Box (composición)

Palabras clave

- `implements`
- interfaces
- composición
- Strategy
- Callback
- Dependency Injection
- pasar objetos al framework

```
public interface Animal { </> Java  
  
    void hacerSonido();  
  
    int edad();  
  
}  
  
public abstract class Perro implements Animal {  
  
    @Override  
    public void hacerSonido() {  
        System.out.println("Guau");  
    }  
  
    @Override  
    public int edad() {  
        return 5;  
    }  
}
```

```
public abstract class Persona {  
    protected String nombre;  
    public Persona(String nombre) {  
        this.nombre = nombre;  
    }  
    class Alumno extends Persona {  
        protected int edad;  
        public Alumno(String nombre) {  
            super(nombre);  
            this.edad = edad;  
        }  
    }  
}
```

```
Alumno a = new Alumno("Nahuel", 21);
```

Patrones de disenho

Adapter

Composite

TemplateMethod

Strategy

State

Builder

Este patrón pertenece a la categoría de **patrones estructurales**, que son aquellos que se enfocan en cómo organizar y relacionar clases / objetos para formar estructuras más grandes y flexibles.

Explicaciones
Patrones

Decorator

FactoryMethod

Proxy

- **Patrones creacionales:** se ocupan de cómo crear objetos.
 - Pregunta que responden: *¿Cómo instancio un objeto de forma flexible?*
 - Ejemplos: Factory Method, Builder.
- **Patrones estructurales:** se ocupan de cómo organizar y relacionar clases u objetos.
 - Pregunta que responden: *¿Cómo hago que estos objetos trabajen juntos?*
 - Ejemplos: Adapter, Composite, Decorator, Proxy.
- **Patrones de comportamiento:** se ocupan de cómo interactúan y colaboran los objetos, o cómo se distribuyen las responsabilidades.
 - Pregunta que responden: *¿Cómo se comunican o cómo ejecutan una tarea entre ellos?*
 - Ejemplos: **Strategy, Template Method, Observer.**

Patrones de diseño

02

1. Agregación (◊ rombo blanco)

👉 Los objetos pueden existir por separado.

Si el objeto "contenedor" desaparece, el otro sigue existiendo.

Universidad ◊ — Profesor

Si cierra la universidad...

El profesor sigue existiendo.

- Si el equipo desaparece...

Los jugadores siguen existiendo.

Equipo ◊ — Jugador

- El equipo tiene jugadores.
- Si el equipo desaparece...

```
<> Java
```

```
Equipo boca = null;
```

Los jugadores siguen existiendo.

```
<> Java
```

```
Jugador messi = new Jugador();
```

Messi puede pasar a otro equipo.

Relación:

Equipo ◊ — Jugador

2. Composición (◆ rombo negro)

👉 El objeto contenido depende del contenedor.

Si el objeto principal muere...

Los objetos internos también.

Auto ◆ — Motor

Casa ◆ — Habitación

Las habitaciones forman parte de la casa.

No existen "sueltas" fuera de ella.

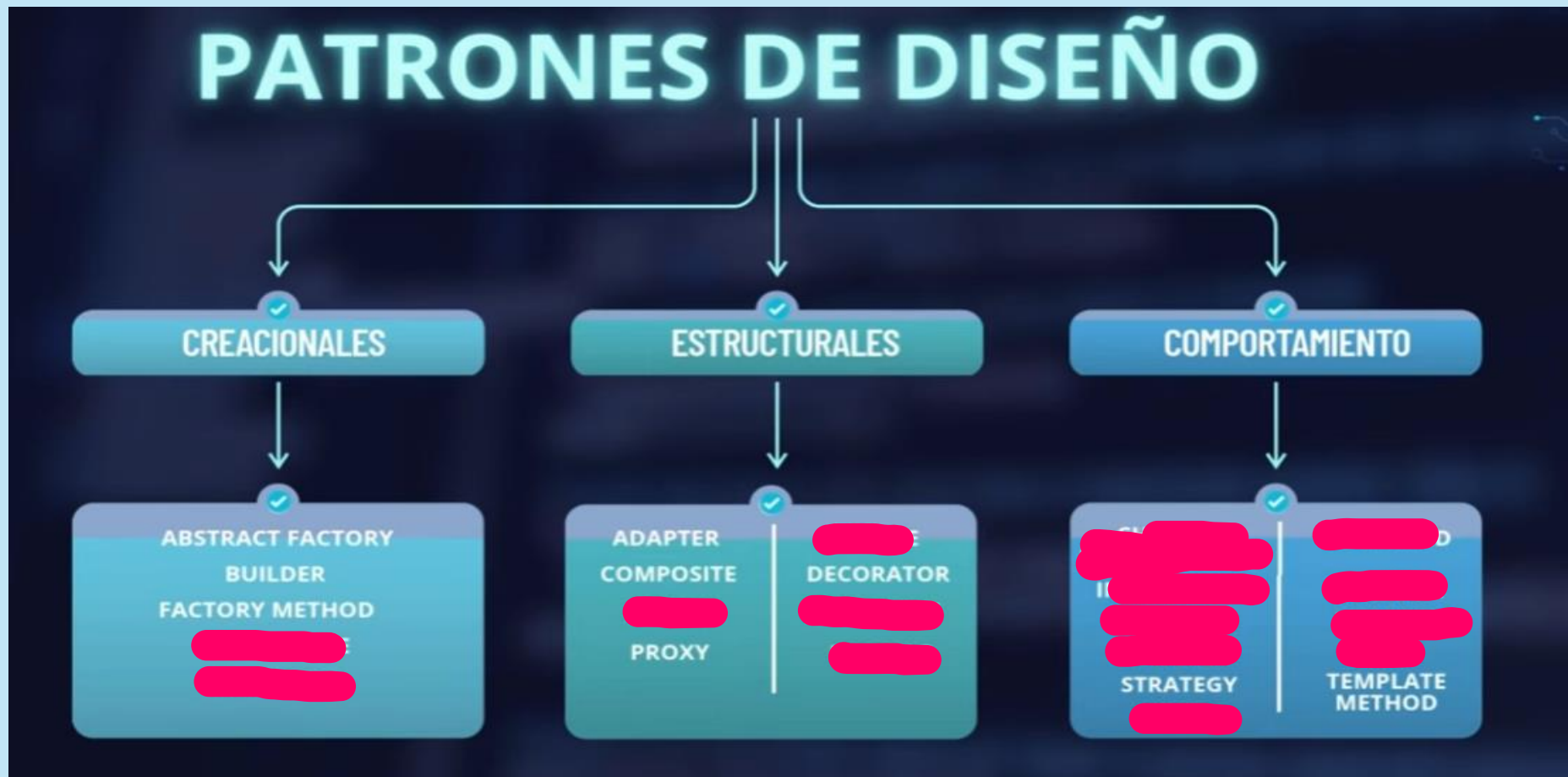
Regla para el parcial

Primero identificá qué tipo de relación hay:

- ◆ → Composición
- ◊ → Agregación
- Sin rombo → Asociación

Después agregás la **cardinalidad**:

- 1
- 0..1
- * (o 0..*)
- 1..*



1. Patrones Creacionales

👉 Se preocupan por cómo se crean los objetos.

2. Patrones Estructurales

👉 Se preocupan por cómo se relacionan o conectan los objetos y clases.

3

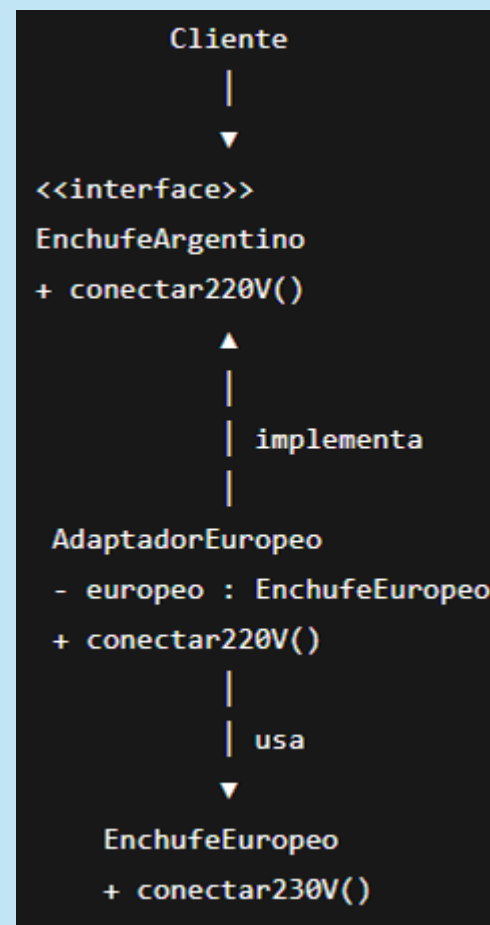
Patrones de comportamiento: se ocupan de cómo interactúan y colaboran los objetos, o cómo se distribuyen las responsabilidades.

Patrones de disenho

Adapter

Patron	Intencion	Aplicabilidad	Tips para saber cuando aplicarla
Adapter	"Convertir" la interfaz de una clase en otra que el cliente espera. Adapter permite que ciertas clases con interfaces incompatibles puedan trabajar en conjunto	Se usa cuando tenemos una interfaz incompatible, es decir cuando tenemos una clase distinta a la que el codigo espera.	No se puede modificar el codigo de ninguna otra clase Vamos a necesitar un Adapter de por medio.

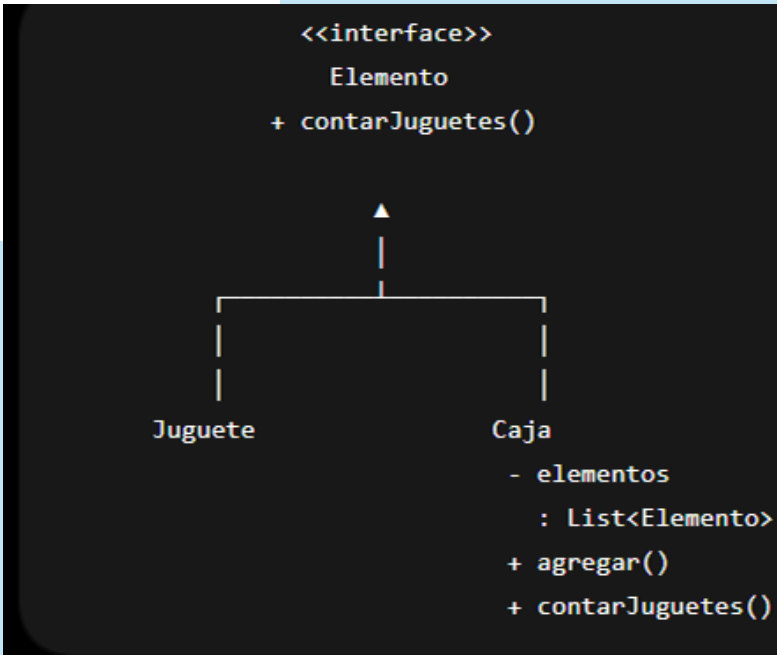
El patrón Adapter se encarga de adaptar una clase existente a una interfaz esperada, sin modificar las clases ya existentes. Esto se logra creando una clase concreta intermedia (Adapter) que mediante composición contiene la clase incompatible y permite acceder a su funcionalidad implementando la interfaz requerida.



Patrones de disenho

Composite

Patron	Intencion	Aplicabilidad	Tips para saber cuando aplicarla
Composite	Componer objetos en estructuras de arbol para representar jerarquias parte-todo. Permite que los clientes traten a los objetos atomicos y a sus composiciones uniformemente.	Usar el patron cuando:: Se quiere presentar jerarquias parte-todo de objetos Se quiere que los objetos "clientes" puedan ignorar las diferencias entre composiciones y objetos individuales.	Podemos aplicar este patron cuando tenemos elementos que pueden contener otros elementos del mismo tipo.



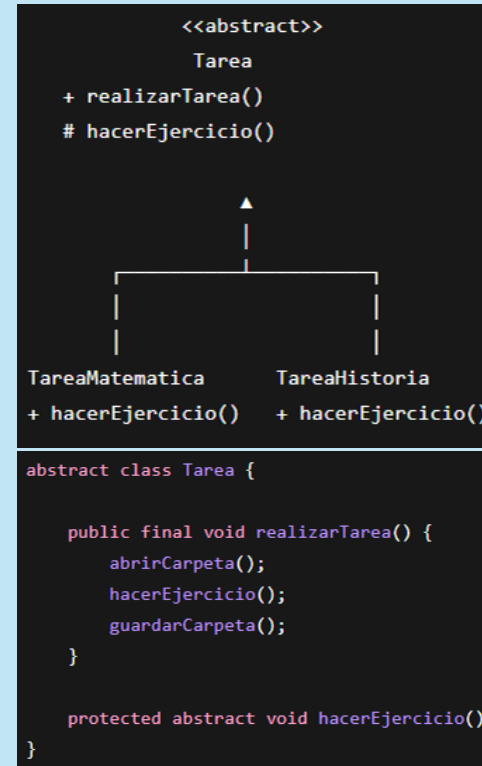
La clave está en que **Caja contiene una lista de Elemento**, no una lista de Juguete ni una lista de Caja.

El patron composite es utilizado cuando se quiere modelar estructuras en forma de arbol para representar una jerarquia, donde uno de los hijos representa una clase individual y otra representa a un grupo de elementos del tipo padre

Patrones de disenho

Patron	Intencion	Aplicabilidad	Tips para saber cuando aplicarla
Template Method	Define el esqueleto de un algoritmo en un metodo de una clase base. Permite que las subclases redefinan ciertos pasos de un algoritmo sin cambiar la estructura general.	Cuando hay varias clases que comparten la logica en general, pero cada una necesitapersonalizar algunos pasos en especifico. Se usa para evitar duplicacion de codigo en las subclases. Se aplican varios metodos comunes dentro de un mismo metodo en la superclase.	Template Method reúne varios metodos dentro de uno mismo que todas las subclases lo implementa, pero los metodos internos pueden redefinirse Si encontramos codigo duplicado en las subclases, posiblemente se pueda aplicar template method. Se define el algoritmo en la superclase. Las subclases implementan pasos especificos.

TemplateMethod



Visualmente

realizarTarea()

1. Abrir carpeta <- fijo
2. Hacer ejercicio <- cambia
3. Guardar carpeta <- fijo

Matemática:

Abrir carpeta
Resolver cuentas
Guardar carpeta

Historia:

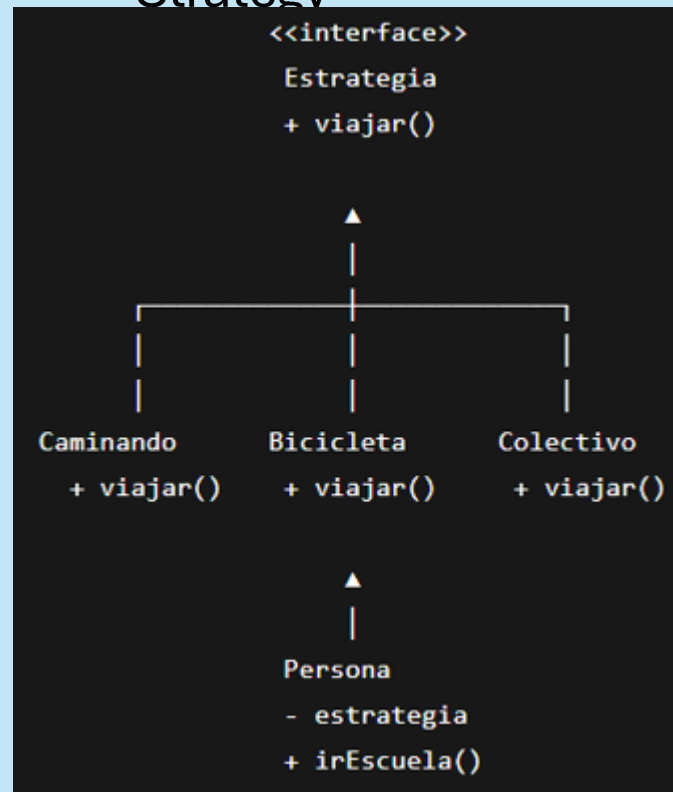
Abrir carpeta
Responder preguntas
Guardar carpeta

El template method se encarga de definir el esqueleto de un algoritmo en la clase base, dejando que algunas etapas sean implementadas o redefinidas por las subclases. De Esta forma se reutiliza la estructura general del proceso sin duplicar codigo.

Patrones de disenho

Strategy

Patron	Intencion	Aplicabilidad	Tips para saber cuando aplicarla
Strategy	Permite encapsular cada algoritmo en un objeto y usarlos en forma intercambiable en tiempo de ejecucion segun se necesiten Permite cambiar en forma dinamica.	No es deseable codificarlos todos en una clase y seleccionar cual utilizar por medio de sentencias condicionales. Es necesario cambiar el algoritmo en forma dinamica, en tiempo de ejecucion.	Cuando tenemos varias estrategias/formas para realizar una misma tarea, podemos usar este patron. Ej : Tarea: Comprimir archivo Estrategias para comprimir: ZIP,RAR,TAR...etc Son distintas estrategias/formas de realizar una tarea (comprimir)



Ejemplo sencillo: Métodos de pago

Tenemos distintas formas de pagar:

`</>` Java

```
interface MetodoPago {
    void pagar(double monto);
}
```

Patrones de disenho

State

Patron	Intencion	Aplicabilidad	Tips para saber cuando aplicarla
State	Permite que un objeto cambie su comportamiento cuando cambia su estado interno. Permitiendo cambiar de estado durante tiempo de ejecucion.	El comportamiento de un objeto, depende de su estado y puede cambiar durante su ciclo de vida. Se quiere cambiar el comportamiento de manera dinamica.	Si el objeto cambia de estado, o tiene un atributo estado, es una señal de que podemos aplicar el patron State, evitamos tener varios if o switch.

```
public class CuentaBancaria {  
    private double saldo;  
    private EstadoCuenta estado;  
  
    public CuentaBancaria() {  
        saldo = 0;  
        estado = new Activa(this);  
    }  
}
```

```
public void cambiarEstado (String nuevo){  
    estado.cambiarEstado (nuevo);  
}
```

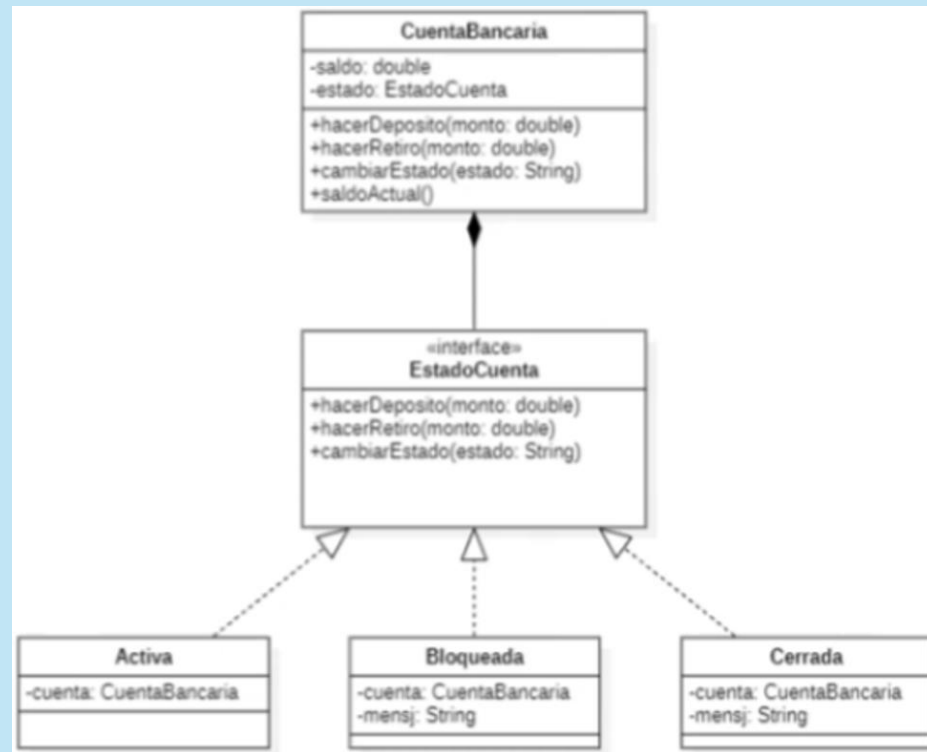
```
public class Activa implements EstadoCuenta {  
    private CuentaBancaria cuenta;  
  
    public Activa (CuentaBancaria cuenta) {  
        this.cuenta = cuenta;  
    }  
  
    @Override  
    public void hacerDeposito (double monto) {  
        cuenta.setSaldo(cuenta.getSaldo() + monto);  
    }  
  
    @Override  
    public void hacerRetiro(double monto) {  
        cuenta.setSaldo(cuenta.getSaldo() - monto);  
    }  
  
    @Override  
    public void cambiarEstado (String estado) {  
        if (estado.equals("cerrar")) {  
            cuenta.setEstado(new Cerrada(cuenta));  
            System.out.println("La cuenta fue cerrada...");  
        } else {  
            cuenta.setEstado (new Bloqueada(cuenta));  
            System.out.println("La cuenta fue bloqueada...");  
        }  
    }  
}
```

```
public interface State {  
    void pressPlay(Player player);  
}
```

```
public class PlayingState implements State {  
    @Override  
    public void pressPlay(Player player) {  
        System.out.println("Ya está sonando. Pausando la música...");  
        player.setState(new PausedState());  
    }  
}
```

```
public class PausedState implements State {  
    @Override  
    public void pressPlay(Player player) {  
        System.out.println("Reanudando la reproducción...");  
        player.setState(new PlayingState());  
    }  
}
```

```
public class Player {  
    private State currentState;  
  
    public Player() {  
        // Estado inicial  
        this.currentState = new PausedState();  
    }  
  
    public void setState(State state) {  
        this.currentState = state;  
    }  
  
    public void clickPlayButton() {  
        // Delega el comportamiento al estado actual  
        currentState.pressPlay(this);  
    }  
}
```



```
class Computadora {
```

```
    private String cpu;
    private int ram;
    private int disco;
    private boolean wifi;
    private boolean bluetooth;
```

```
    private Computadora(Builder builder) {
        this.cpu = builder.cpu;
        this.ram = builder.ram;
        this.disco = builder.disco;
        this.wifi = builder.wifi;
        this.bluetooth = builder.bluetooth;
    }
```

```
static class Builder {
```

```
    private String cpu;
    private int ram;
    private int disco;
    private boolean wifi;
    private boolean bluetooth;
```

```
    public Builder cpu(String cpu) {
        this.cpu = cpu;
        return this;
    }
```

```
    public Builder ram(int ram) {
        this.ram = ram;
        return this;
    }
```

```
    public Builder disco(int disco) {
        this.disco = disco;
        return this;
    }
```

```
    public Builder wifi(boolean wifi) {
        this.wifi = wifi;
        return this;
    }
```

```
    public Builder bluetooth(boolean bluetooth) {
        this.bluetooth = bluetooth;
        return this;
    }
```

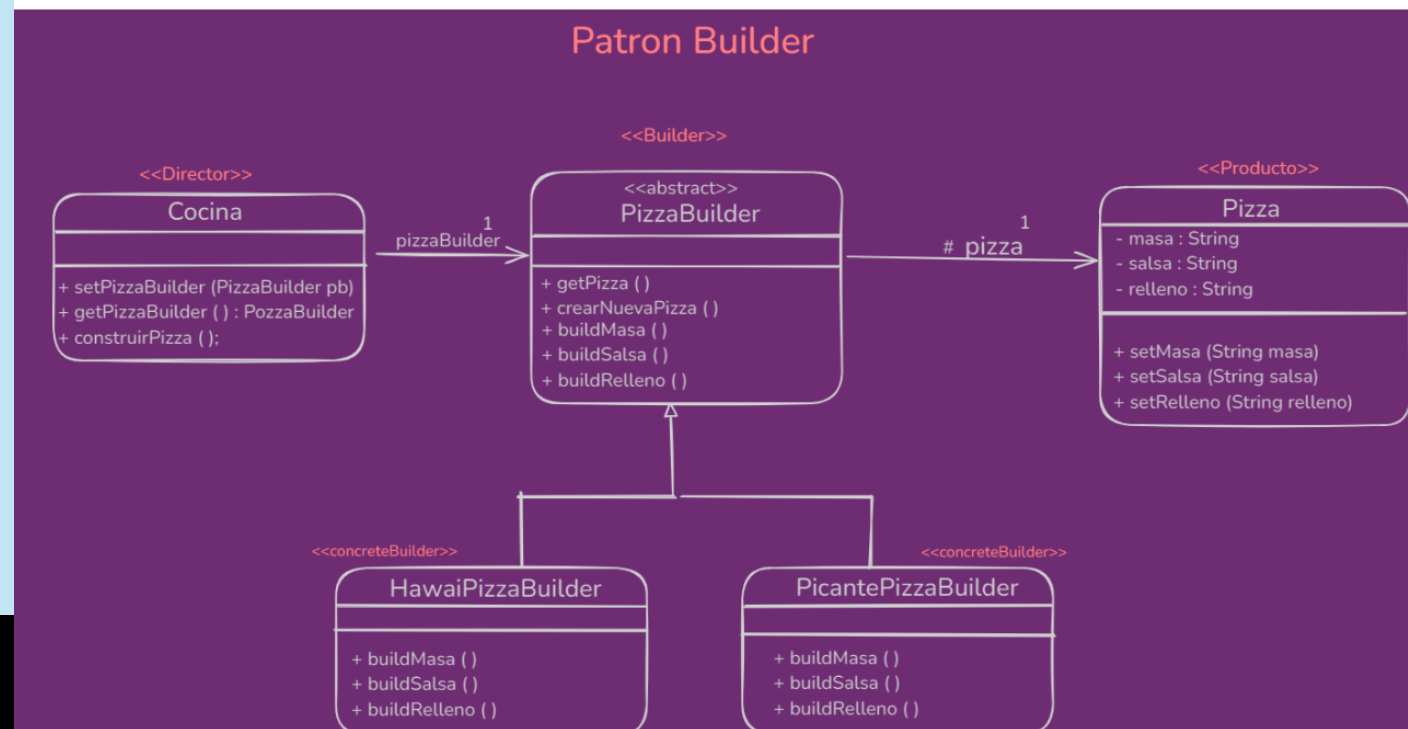
```
    public Computadora build() {
        return new Computadora(this);
    }
```

```
}
```

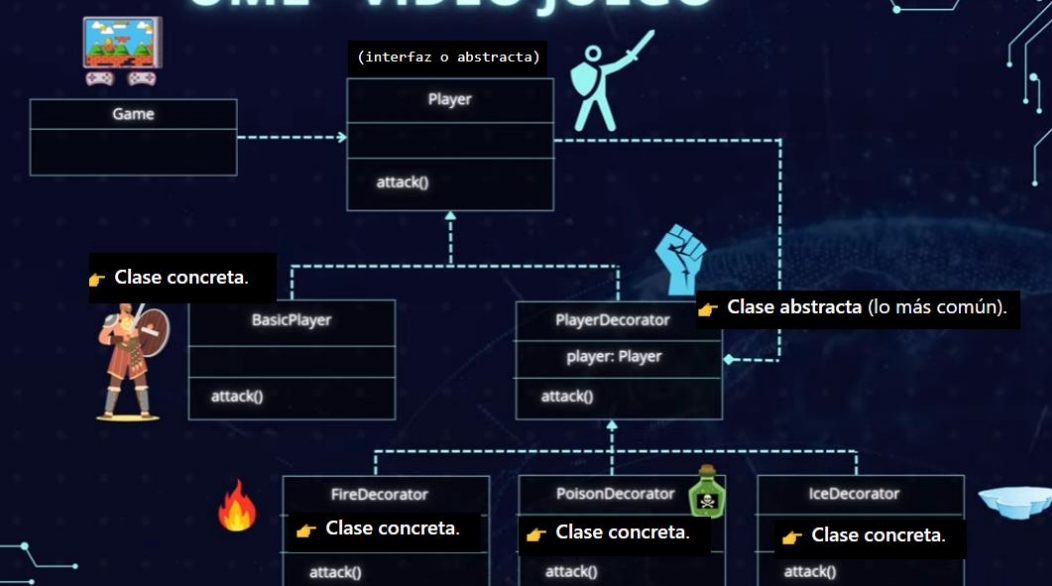
Patrones de disenho

Builder

Ejemplo de Builder con Director, Producto, Builder, ConcretClass



```
Computadora pc = new Computadora.Builder()
    .cpu("Intel i5")
    .ram(16)
    .disco(512)
    .wifi(true)
    .bluetooth(false)
    .build();
```



```

public class Main {

    public static void main(String[] args) {

        // Objeto original
        Cafe cafe = new CafeSimple();

        System.out.println(cafe.descripcion());
        // Café

        // Agrego leche
        cafe = new LecheDecorator(cafe);

        System.out.println(cafe.descripcion());
        // Café + leche

        // Agrego azúcar
        cafe = new AzucarDecorator(cafe);

        System.out.println(cafe.descripcion());
        // Café + leche + azúcar

    }
}
  
```

Patrones de disenho

Decorator

</> Java

```

interface Cafe {
    String descripcion();
}
  
```

</> Java

```

class CafeSimple implements Cafe {
    public String descripcion() {
        return "Cafe";
    }
}
  
```

```

abstract class Decorator implements Cafe {

    protected Cafe cafe;

    public Decorator(Cafe cafe) {
        this.cafe = cafe;
    }
}
  
```

```

class LecheDecorator extends Decorator {

    public LecheDecorator(Cafe cafe) {
        super(cafe);
    }

    @Override
    public String descripcion() {
        return cafe.descripcion() + " + leche";
    }
}
  
```

```

class AzucarDecorator extends Decorator {

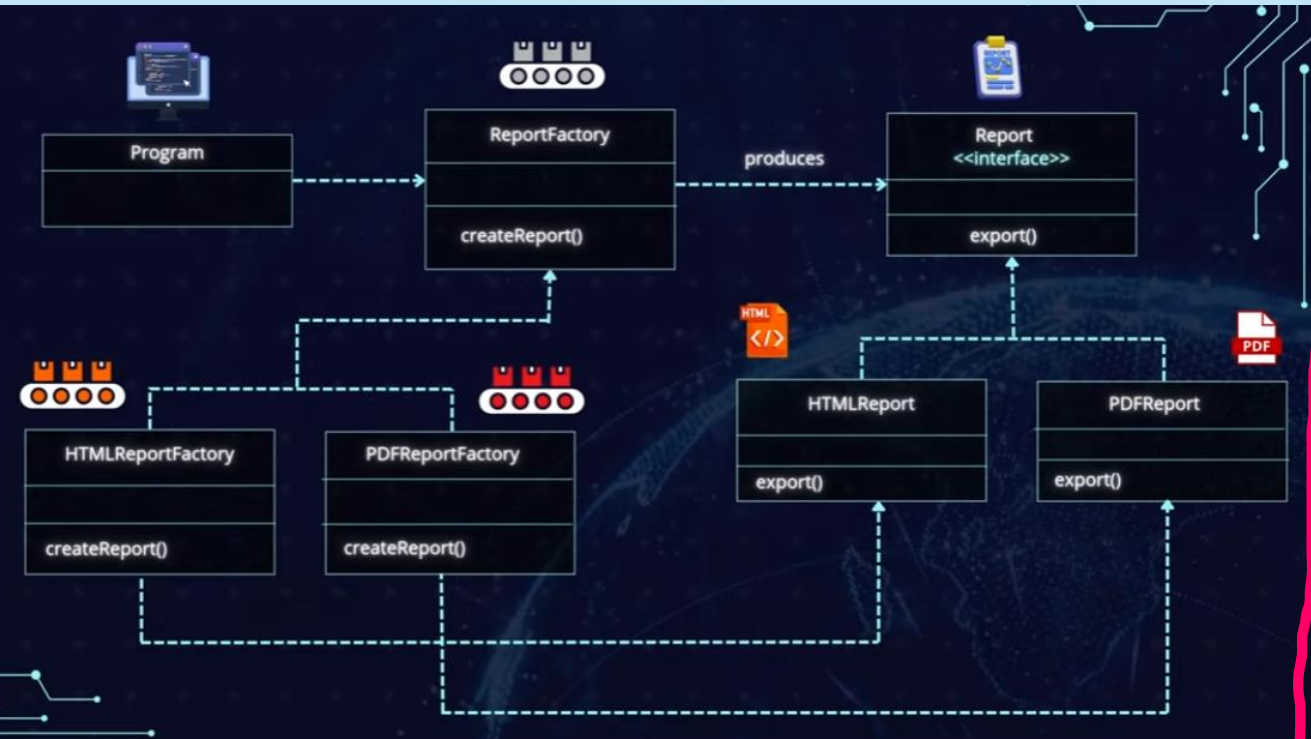
    public AzucarDecorator(Cafe cafe) {
        super(cafe);
    }

    @Override
    public String descripcion() {
        return cafe.descripcion() + " + azúcar";
    }
}
  
```

Un decorador envuelve a otro objeto del mismo tipo (interfaz o clase abstracta), por lo que se pueden encadenar múltiples decoradores para agregar funcionalidades de forma dinámica.

Patrones de disenho

FactoryMethod



```
interface Report {  
  exportar(): void;  
}
```

```
class HTMLReport implements Report {  
  export() {  
    console.log("Exporting HTML Report");  
  }  
}
```

```
class PDFReport implements Report {  
  export() {  
    console.log("Exporting PDF Report");  
  }  
}
```

```
abstract class ReportFactory {  
  abstract createReport(): Report;  
}
```

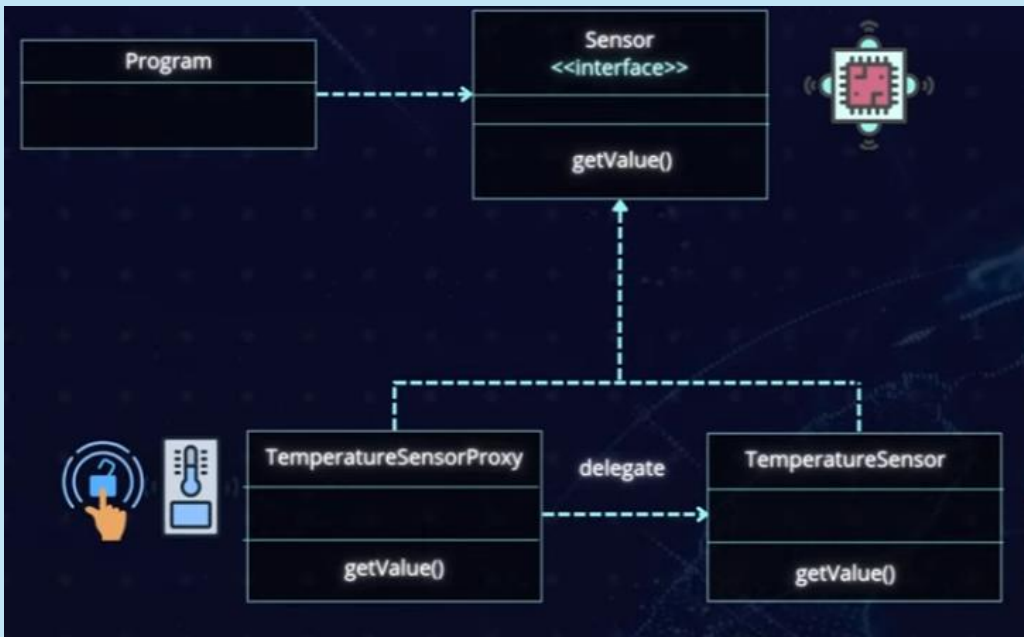
```
class HTMLReportFactory extends ReportFactory {  
  createReport() {  
    return new HTMLReport();  
  }  
}
```

```
class PDFReportFactory extends ReportFactory {  
  createReport() {  
    return new PDFReport();  
  }  
}
```

```
class Program {  
  constructor(private factory: ReportFactory) {}  
  
  run() {  
    const report = this.factory.createReport();  
    report.export();  
  }  
}
```

Patrones de disenho

Proxy



```
interface Sensor {
  getValue(): number;
}
```

```
class TemperatureSensor implements Sensor {
  getValue(): number {
    return 24 + Math.random() * 4;
  }
}
```

```
class TemperatureSensorProxy implements Sensor {
  private sensor: TemperatureSensor | null = null;

  constructor(private role: string) {}

  getValue(): number {
    if (this.role !== "admin") {
      throw new Error("Access denied!");
    }

    if (!this.sensor) {
      this.sensor = new TemperatureSensor();
    }

    return this.sensor.getValue();
  }
}
```

```
const sensor1 = new
TemperatureSensorProxy("admin");
const temp1 = sensor1.getValue(); // Gets the
temperature
```

```
const sensor2 = new TemperatureSensorProxy("non-
admin");
const temp2 = sensor2.getValue(); // throws an
error
```

Guía Maestra de Patrones de Diseño: Estructuras y Propósitos UML

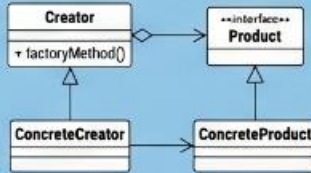
Referencia rápida sobre la encapsulación de la creación, composición y comportamiento de objetos para la flexibilidad y mantenimiento del código fuente.

PATRONES CREACIONALES Y ESTRUCTURALES

Factory Method (Método de Fabricación)

Define una interfaz para crear un objeto, pero delega la instanciación a las subclases.

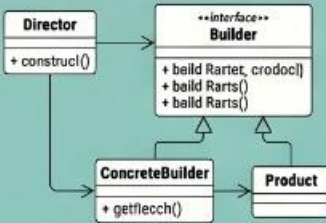
Encapsula la lógica de instanciación en un único lugar, mitigando problemas de jerarquías paralelas.



Builder (Constructor)

Separa la construcción de un objeto complejo de su representación (implementación).

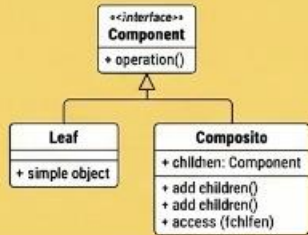
El Director conoce los pasos; el Builder construye las partes y el Producto es el resultado final.



Composite (Compuesto)

Compone objetos en estructuras de árbol para representar jerarquías de "parte-todo".

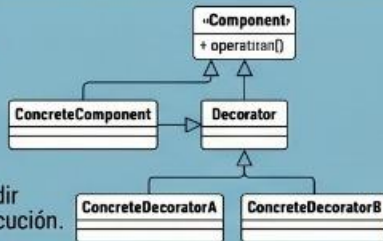
Permite que el cliente trate a los objetos individuales y a las composiciones de forma uniforme.



Decorator (Decorador)

Agrega responsabilidades a un objeto de manera dinámica y transparente sin usar herencia estática.

Funciona como un "wrapper" que envuelve al componente para añadir comportamiento en tiempo de ejecución.



Resumen Comparativo: Construcción

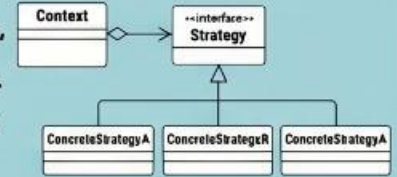
Característica	Factory Method	Builder
Problema	Construcción de objetos	Construcción de estructuras complejas
Encapsula complejidad	Sí	Sí
Identifica proveedor de partes	No	Sí

PATRONES DE COMPORTAMIENTO Y CONECTIVIDAD

Strategy (Estrategia)

Define una familia de algoritmos, los encapsula y los hace intercambiables dinámicamente.

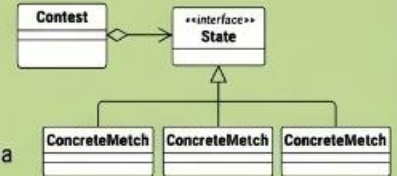
El Cliente suele crear la estrategia y el Contexto la utiliza mediante una interfaz común.



State (Estado)

Permite que un objeto cambie su comportamiento cuando cambia su estado interno.

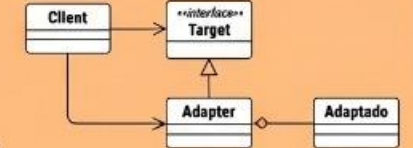
Desacopla el estado en una jerarquía de clases donde cada una representa un estado concreto.



Adapter (Adaptador)

Permite que clases con interfaces incompatibles trabajen juntas.

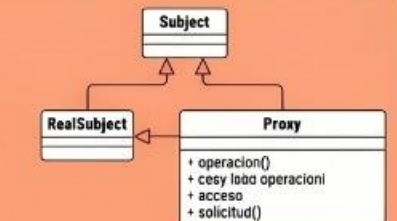
A diferencia del Decorador, el Adaptador cambia la interfaz del objeto original.



Proxy (Sustituto)

Proporciona un representante o intermediario para otro objeto para controlar el acceso a él.

Mantiene una interfaz idéntica al objeto que representa para ser transparente al cliente.



REFACTORING

REFACTORINGS

BADSMALLS

1) Detectar smells

↓

2) Eliminar duplicación sencilla
(Extract Method)

↓

3) Si hay Type Code

Crear jerarquía
(Subclass / Strategy / State)

↓

4) Reemplazar switch
(Replace Conditional with Polymorphism)

5) Mover responsabilidades
(Move Method)

↓

6) Encapsular campos

↓

7) Resolver Primitive Obsession

↓

8) Eliminar Dead Code

↓

9) Reemplazar Magic Numbers

Duplicate Code

Duplicate Code= Extract method

Hay una secuencia de instrucciones repetidas

Switch Statement=

El valor depende del valor de tipo

Una regla que puede servirte para el parcial

Una regla que puede servirte para el parcial

Ct

• Cuando veas:

- • ◆ Código repetido → **Extract Method** (si está dentro de la misma clase).
- • ◆ Código repetido entre varias clases con pequeñas diferencias → **Template Method** o **Pull Up Method**, según el contexto.
- • ◆ Un método usa demasiado otra clase → **Move Method**.
- • ◆ Muchos `if` o `switch` según un tipo → **Replace Conditional with Polymorphism**

REFACTORING

Duplicate Code

REFACTORINGS

BADSMALLS

Duplicate Code= Extract method
Hay una secuencia de instrucciones repetidas

Switch or IFS= Replace Conditional With Polymorphism
Hay una secuencia de instrucciones repetidas

Respuesta de parcial

1. Code smell: Type Code (o Switch Statements).

2. ¿Por qué?

Porque el comportamiento depende del atributo `tipo` mediante un `switch`. Cada vez que se agregue un nuevo tipo de producto habrá que modificar el método `calcular()`.

3. Refactoring:

Replace Conditional with Polymorphism.

4. Resultado:

Cada subtipo de `Producto` implementa su propio método `calcularPrecio()`, eliminando el `switch` y delegando el comportamiento al objeto correspondiente.

✦ Un resumen que te puede servir para reconocer estos ejercicios:

Lo que ves	Smell	Refactoring
Código repetido	Duplicate Code	Extract Method
Un método usa muchos datos de otra clase	Feature Envy	Move Method
<code>if / switch</code> según un tipo	Type Code / Switch Statements	Replace Conditional with Polymorphism
Campos públicos	Data Class / Mala encapsulación	Encapsulate Field

REFACTORING

Feature Envy



Move Method

```
class Factura {
    private Cliente cliente;
    public String obtenerDatosCliente(){
        return cliente.getNombre()
        + " - "
        + cliente.getDireccion()
        + " - "
        + cliente.getTelefono();
    }
}

class Reporte {
    public void imprimirFactura(Factura f){
        System.out.println(
            f.getCliente().getNombre()
            + f.getCliente().getDireccion()
            + f.getCliente().getTelefono()
        );
    }
}
```

```
1. ¿Un método usa muchos datos de otra clase?
    cliente.getNombre()
    cliente.getDireccion()
    cliente.getTelefono()

    ↓

SMELL
✓ Feature Envy

    ↓

REFACTORING
✓ Move Method

    ↓

La responsabilidad pasa al objeto
que posee esos datos.
```

REFACTORING

DuplicatedCode



Extract Method

```
class Usuario {  
    void registrar(){  
        validarEmail();  
        guardarBD();  
        enviarEmailBienvenida();  
    }  
  
    void actualizar(){  
        validarEmail();  
        guardarBD();  
        enviarEmailBienvenida();  
    }  
}
```

■ 2. ¿Hay código repetido?

```
registrar()  
actualizar()
```

hacen exactamente lo mismo.

↓

SMELL

✓ Duplicate Code

↓

REFACTORING

✓ Extract Method

↓

Crear un método común y llamarlo
desde ambos.

Cuando ves código:

1) ¿Hay código repetido?

→ Extract Method

2) ¿Un método usa mucho otra clase?

→ Move Method

3) ¿Hay if/switch por tipo?

→ Polimorfismo

4) ¿Hay atributos públicos?

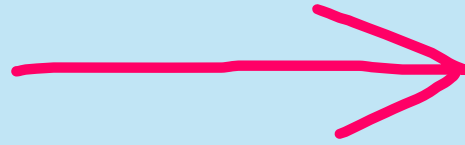
→ Encapsulate Field

5) ¿Hay int/string representando objetos?

→ Primitive Obsession

REFACTORING

Switch Statement



Replace Conditional With Polymorphism

```
class CalculadoraPrecio {  
    double calcular(Producto p){  
        switch(p.tipo){  
            case 1:  
                return p.precio * 0.9;  
  
            case 2:  
                return p.precio * 0.8;  
  
            case 3:  
                return p.precio * 0.5;  
        }  
    }  
}
```

■ 3. ¿Hay if/switch según un tipo?

switch(tipo)

if(tipo==1)

if(tipo==2)

↓

SMELL

✓ Switch Statement

✓ Type Code

↓

REFACTORING

✓ Replace Conditional with Polymorphism

↓

Crear subclases y que cada una
implemente su comportamiento.

REFACTORING

Public Field

```
class Persona {  
    public String nombre;  
    public int edad;  
}
```

Encapsulate Field

■ 4. ¿Los atributos son públicos?

```
public String nombre;
```

```
public int edad;
```

↓

SMELL

✓ Public Field

↓

REFACTORING

✓ Encapsulate Field

↓

private + getters/setters

REFACTORING

Primitive Obsession

```
class Usuario {  
    Private String nombre;  
    Private String calle;  
    Private String ciudad;  
    Private String provincia;  
}
```

Replace Data Value With Object

```
■ 5. ¿Muchos String/int representan un concepto?  
  
String calle;  
String ciudad;  
String provincia;  
  
↓  
  
SMELL  
✓ Primitive Obsession  
  
↓  
  
REFACTORING  
✓ Replace Data Value with Object  
  
↓  
  
Crear una clase Direccion.
```

REFACTORING

Magic number

```
class Banco {  
    boolean puedeRetirar(double saldo){  
        if(saldo > 50000){  
            return true;  
        }  
    }  
}
```

Replace Magic Number With Constant

```
■ 6. ¿Hay números "mágicos"?  
  
if(saldo > 50000)  
  
    ↓  
  
SMELL  
✓ Magic Number  
  
    ↓  
  
REFACTORING  
✓ Replace Magic Number with Constant  
  
    ↓  
  
static final double LIMITE = 50000;
```

REFACTORING

Long Method

```
class Pedido {  
    void procesar(){  
        validarCliente();  
        calcularPrecio();  
        aplicarDescuento();  
        generarFactura();  
        enviarMail();  
        actualizarStock();  
        guardarHistorial();  
    }  
}
```

7. ¿El método tiene 200 líneas?

```
procesar(){
```

```
...
```

```
...
```

```
...
```

```
}
```

↓

SMELL

✓ Long Method

↓

REFACTORING

✓ Extract Method

↓

Dividir en métodos pequeños.

REFACTORING

Type Code + Polimorfismo

```
class Empleado {  
    int tipo;  
    double sueldo(){  
        if(tipo==1){  
            return 100000;  
        }  
        if(tipo==2){  
            return 150000;  
        }  
        if(tipo==3){  
            return 200000;  
        }  
    }  
}
```

8. ¿Hay un atributo "tipo" que decide todo?

```
int tipo;
```

```
if(tipo==1)...
```

```
if(tipo==2)...
```

↓

SMELL

✓ Type Code

↓

REFACTORING

✓ Replace Type Code

✓ Polymorphism

↓

EmpleadoTiempoCompleto

EmpleadoContratado

etc.

REFACTORING

Dead Code

```
class Sistema {  
    void iniciar(){  
        cargarDatos();  
    }  
    void probar(){  
        System.out.println("hola");  
    }  
    int calcular(){  
        return 10;  
    }  
}
```

probar() nunca se llama.

```
■ 9. ¿Hay métodos que nunca se usan?  
  
void probar(){  
  
}  
  
↓  
  
SMELL  
✓ Dead Code  
  
↓  
  
REFACTORING  
✓ Remove Dead Code  
  
↓  
  
Eliminar el código muerto.
```

REFACTORING

¿Código repetido?



Extract Method

¿Usa demasiado otra clase?



Move Method

¿Hay if/switch por tipo?



Polimorfismo

¿Campos públicos?



Encapsulate Field

¿Muchos String/int representan un objeto?



Replace Data Value with Object

¿Número mágico?



Replace Magic Number with Constant

¿Método enorme?



Extract Method

¿Código que nunca se ejecuta?



Remove Dead Code

REFACTORING

Aaaa

```
class Triangulo {  
    double base;  
    double altura;  
}  
class Calculadora {  
    double calcularArea(Triangulo t){  
        return (t.base * t.altura) / 2;  
    }  
}
```

Diferencia de transición: State vs Strategy

En State la transición ocurre entre estados; en Strategy suele cambiarse por acción del cliente mediante un setter.

Fijate que tiene:

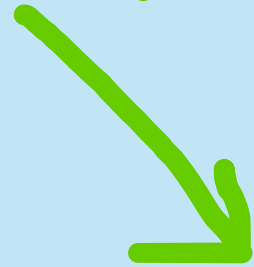
📄 Java

```
protected Cafe cafe;
```

Eso es composición.

Frameworks

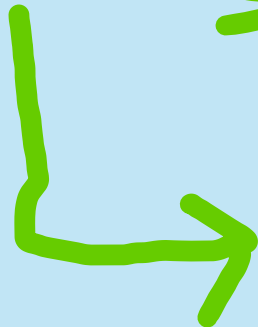
Framework → FrozenSpot



HotSpot



BlackBox (Composicion/interfaces/configuracion/~~clases~~)



WhiteBox (Herencia / tiene metodos abstractos)

Un método es un hotspot cuando el framework espera que el desarrollador lo implemente o lo sobrescriba para personalizar su comportamiento.

No cualquier método sobrescribible es un hotspot: debe estar pensado como un punto de extensión del framework.

N/S

¿Es un HotSpot?
¿Es White Box o Black Box?
¿Hay Control de Inversión? ¿Dónde?
¿Se modifica el framework?

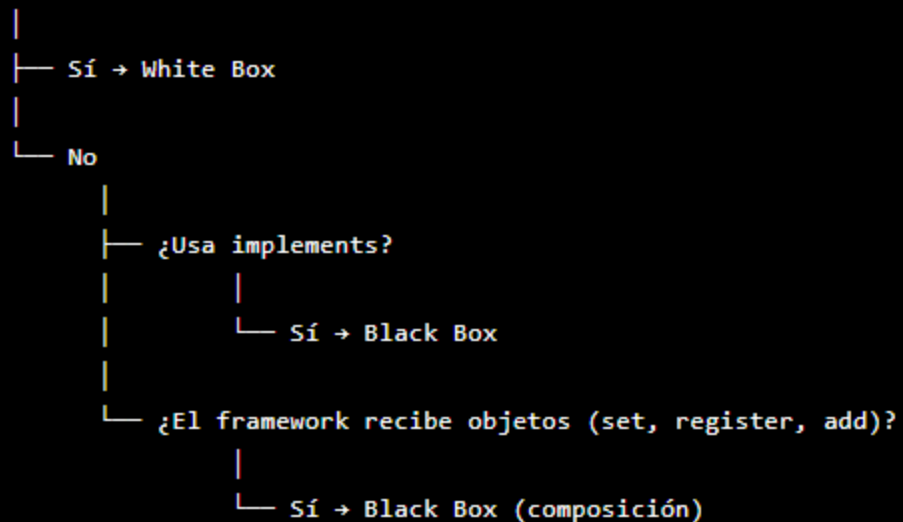
Regla de oro para el parcial

Cada vez que veas un ejercicio, hacete estas preguntas en este orden:

1. ¿Estoy creando una clase nueva para el framework? → Sí ⇒ Hay HotSpot.
2. ¿Uso `extends`? → Sí ⇒ White Box.
3. ¿Uso `implements` o composición? → Sí ⇒ Black Box.
4. ¿Quién llama al método sobrescrito? Si lo llama el framework ⇒ Control de Inversión (IoC).
5. ¿Tuve que modificar código del framework? → No ⇒ Está preparado para extenderse (principio Open/Closed).

`MercadoPago` representa un HotSpot del framework porque agrega un nuevo método de pago sin modificar el framework. Es un framework White Box porque la extensión se realiza mediante herencia (`extends PaymentMethod`). La llamada a `registerPaymentMethod()` registra la nueva implementación. El Control de Inversión ocurre cuando el framework decide invocar el método `pay()` durante el proceso de pago.

¿La clase usa extends?



Luego preguntate:

¿Ese código lo escribo yo?

Sí → HotSpot

No → Frozen Spot

Y por último:

¿Quién llama al método?

Yo → No hay IoC

El framework → Sí hay IoC

Frameworks

lo primero es ver si te piden algo que involucre una jerarquía de subclases, corte Template Method, Strategy, o State, y si te piden eso aplicar **Replace Type Code with Subclasses/Strategy/State** para extraer las subclases, y después para repartir un condicional entre las subclases el **Replace Conditional with Polymorphism**.

yo te recomiendo x las dudas igual que leas todos los pasos del refactoring en el libro y q trates de hacerlos a tu manera por lo menos

Framework de caja negra vs caja blanca:

Framework de caja negra: Es aquel donde no se ve ni se puede modificar el loop de control. Simplemente se siguen las reglas para utilizar el framework y definimos lo que nos permitan. En este tipo de frameworks **no sabemos cuando ni como se llama el código**. No sabemos ni modificamos el flujo interno, solo podemos "enchufar" componentes.

Framework de caja blanca: Tenemos visibilidad y control sobre el flujo del framework (loop de control), por ejemplo podemos extender clases del framework y redefinir metodos.

Frameworks

Una librería

Vos manejás el programa.

Java

```
public static void main(String[] args){  
    MiLibreria.imprimir();  
}
```

Vos llamás a la librería.

Un Framework

El framework maneja el programa.

Vos solamente completás algunas clases.

Ejemplo:

Java

```
public class MiFiltro implements MidiFilter{  
  
    public MidiSignalData process(MidiSignalData data){  
        ...  
    }  
  
}
```

Después el framework hace:

```
Framework  
↓  
llama process()
```

No vos.

Por eso se dice:

"Don't call us, we'll call you."

Eso es la famosa Inversión de Control.

En un framework, yo implemento o extiendo clases que el framework me proporciona. Luego el framework es quien invoca mis métodos cuando corresponde.

WhiteBox

El framework se extiende heredando clases.

Ejemplo

Java

```
class MiDevice extends MidiDevice{  
  
    @Override  
    readSignal(){...}  
  
}
```

Vos modificás el comportamiento mediante herencia.

BlackBox

El framework se configura pasando objetos.

Ejemplo

Java

```
framework.registerFilter(  
    new EchoFilter()  
);
```

No heredás del framework.

Solo le enchufás objetos.

Mucho más flexible.

Frameworks

Frozen

Ya está implementado.

No lo tocás.

Ejemplo

```
</> Java  
  
start()  
stop()  
registerDevice()
```

Todos esos ya existen.

Hot Spot

Son los lugares donde vos agregás comportamiento.

Ejemplo

```
</> Java  
  
readSignal()  
  
process()
```

Porque cada dispositivo lee distinto.

Y cada filtro procesa distinto.

Entonces el usuario implementa esos métodos.

4. Inversión de Control

Otra pregunta fija.

Mirá este código.

```
</> Java  
  
signal = device.readSignal();
```

¿Quién llama?

El framework.

No la aplicación.

Entonces

Sí hay IoC.

En un framework, yo implemento o extiendo clases que el framework me proporciona. Luego el framework es quien invoca mis métodos cuando corresponde.

Frameworks

Lo importante en un Framework

Lo que quieren que entiendas en el parcial es esto:

```
Framework
|
| define interfaces o clases abstractas
▼
Vos implementás o extendés esas clases
▼
El Framework crea el flujo del programa
▼
En algún momento llama a tu método

process(data)
```

A esto se lo conoce como **Inversión de Control (IoC)**: el flujo del programa lo controla el framework, no tu código.

No te estás extendiendo. Estás implementando una interfaz que el framework define.

```
Framework
|
|— interface MidiFilter
|
▼
Tu clase
MiFiltro implements MidiFilter
```

Frameworks

~ Thiago

Gente les dejo un machetito de frameworks por si les sirve

Caja blanca: tiene métodos abstractos

Caja negra: tiene interfaces/clases abstractas

Hot spot: lo abstracto/interfaz

Frozen spot: lo que no es hot spot

Dónde hay inversión de control: en las partes abstractas

Todo lo que lean de teoría es para justificar esto pero por si alguno piensa no dar ese tema en el recu, que se aprenda esto e intente que es una tontería así (después que guitarree para justificar jajaja)

9:33 p

Frameworks

Composite = caja negra 7:16 p.m.

Solo herencia = caja blanca 7:17 p.m.

~ Emanuel

Si ves que al instanciar un objeto, luego le agregan otras instancias, entonces caja negra 7:19 p.m.

De lo contrario, caja blanca 7:19 p.m.

4) ¿Se pueda implementar una tarea que implemente el nivel de urgencia incrementando su valor al acercarse la fecha límite? En base a su respuesta, ¿para que es un trigger aquí o un hot spot?

2) ¿Observa métodos gancho? ¿Cuáles?

3) Analizando la clase Turno, los programadores de OndaDeTareasApp pueden asegurar si hay una inversión de control? ¿Por qué?

buenas, consulta en el 4 de este la respuesta es que no hay inversión de control en la clase turno no?

4:53 p.m.

~ Emanuel

~ Ivan

+54 9 2216 74-6100

buenas, consulta en el 4 de este la respuesta es que no hay inversión de control en la clase turno no?

La inversión de control se produce únicamente cuando el framework invoca a un método desarrollado por el usuario

4:55 p.m.